Unit 6 · Testing and debugging

CONCEPT & REASONING

Find errors systematically, check expected against observed behaviour, and demonstrate that your Python functions handle important cases.

APPLIED EXAMPLE / TAKEAWAY

CONCEPT: tests detect failures
PROCESS: debugging explains and repairs failures

Learning outcomes

CONCEPT & REASONING

Recognise syntax, runtime and logic errors; trace faulty loops and conditions; design normal, boundary and invalid tests; validate input; use assert; and document repairs.

APPLIED EXAMPLE / TAKEAWAY

Practical task: repair and verify a five-day expenditure analyser.

Why test and debug programmes

CONCEPT & REASONING

Testing compares actual behaviour with the requirement. Debugging identifies why a failed case behaves incorrectly. Both improve confidence; neither guarantees every possible scenario.

PYTHON EXAMPLE / CHECK

```
expenses = [2000, 3000, 0, 4000,
1000]
print(sum(expenses))
```

Why test and debug programmes · applied

CONCEPT & REASONING

For [2000,3000,0,4000,1000], the expected total is 10,000 TZS. Write the expected value before pressing Run.

PYTHON EXAMPLE / CHECK

```
Expected: 10000
Observed: 10000
Result: PASS
```

Start with a clear requirement

CONCEPT & REASONING

A clear, checkable requirement specifies inputs, processing and observable outputs. Ambiguous words such as “works well” cannot by themselves define a test.

PYTHON EXAMPLE / CHECK

```
if total > allowance:
    message = "Over budget"
```

Start with a clear requirement · applied

CONCEPT & REASONING

Write rules before building a test: five non-negative accepted values; accurate sum/average/zeros; an exact equality branch; clear re-prompting for invalid entries.

PYTHON EXAMPLE / CHECK

```
Expected output: defined before
execution
```

Three common categories of errors

CONCEPT & REASONING

Syntax: code cannot be parsed. Runtime: an exception occurs during execution. Logic: code runs but violates a requirement.

PYTHON EXAMPLE / CHECK

```
if total > allowance
    print("Over")
```

Three common categories of errors · applied

CONCEPT & REASONING

The programme may display a believable value even when the denominator is wrong. Wrong-but-plausible is a reason to test, not to trust.

PYTHON EXAMPLE / CHECK

```
average = total / 4  # wrong for
5 days
```

Read an error message and traceback

CONCEPT & REASONING

Read the exception type and message; note the file and line; recreate the failure using the same input. Do not randomly edit several lines at once.

PYTHON EXAMPLE / CHECK

```
values = [100, 200]
print(values[2])
```

Read an error message and traceback · applied

CONCEPT & REASONING

For a two-element list, indices 0 and 1 are valid. Accessing index 2 fails. The fix is to use a valid position or adjust the loop bounds.

PYTHON EXAMPLE / CHECK

```
IndexError: list index out of range
```

Reproduce the problem before fixing it

CONCEPT & REASONING

Reproduce → isolate → explain → change one cause → rerun tests. Save the failing input so the error does not return later.

PYTHON EXAMPLE / CHECK

```
total_wrong([10])
```

Reproduce the problem before fixing it · applied

CONCEPT & REASONING

A single amount of 10 should total 10. With range(len(values)-1), no indices are visited, so the faulty function returns 0.

PYTHON EXAMPLE / CHECK

```
Expected: 10 | Actual: 0 | FAIL
```

Trace execution step by step

CONCEPT & REASONING

Trace each iteration and track the changing variable. Do not skip the loop condition on the final attempted iteration.

PYTHON EXAMPLE / CHECK

```
i | value | running total
0 | 10 | 10
1 | 20 | 30
```

Trace execution step by step · applied

CONCEPT & REASONING

For [10,20,30], the faulty loop visits 10 and 20, leaving 30 unprocessed. A step-by-step table reveals exactly what was omitted.

PYTHON EXAMPLE / CHECK

```
Correct total = 60; buggy total
= 30
```

Use carefully chosen print statements

CONCEPT & REASONING

Temporary print statements show what Python really did, not what we assumed it did. Label every displayed value.

PYTHON EXAMPLE / CHECK

```
print("DEBUG day:", i + 1)
print("DEBUG amount:", amount)
```

Use carefully chosen print statements · applied

CONCEPT & REASONING

For loops, inspect iteration number, element and running total. Stop printing after the bug is understood.

PYTHON EXAMPLE / CHECK

```
DEBUG amount: 3000
```

Use a debugger or breakpoint when available

CONCEPT & REASONING

Breakpoints pause a running programme. Step Over executes the current line without entering a called function; Step Into follows a function call.

PYTHON EXAMPLE / CHECK

```
total = 0
# pause here; inspect values
for amount in values:
    total += amount
```

Use a debugger or breakpoint when available · applied

CONCEPT & REASONING

The purpose is to observe state and locate the first moment when an actual value differs from expectation.

PYTHON EXAMPLE / CHECK

```
Pause → inspect → step → explain
```

Write a proper test case

CONCEPT & REASONING

Record test ID, input, expected output, actual output and PASS/FAIL. A test without expected output cannot reveal a logic error reliably.

PYTHON EXAMPLE / CHECK

```
T01 | total five amounts
```

Write a proper test case · applied

CONCEPT & REASONING

Compute the arithmetic outside the faulty function: 2,000 + 3,000 + 0 + 4,000 + 1,000 = 10,000.

PYTHON EXAMPLE / CHECK

```
Expected 10000; actual 9000 → FAIL
```

Normal, boundary and invalid inputs

CONCEPT & REASONING

Normal inputs show common behaviour. Boundary inputs exercise equality, zero and limits. Invalid inputs check that the programme rejects wrong values safely.

PYTHON EXAMPLE / CHECK

```
total = 15000
allowance = 15000
```

Normal, boundary and invalid inputs · applied

CONCEPT & REASONING

A programme might pass a normal case and still fail when total equals allowance. Always include at least one equality case.

PYTHON EXAMPLE / CHECK

```
Correct: Exactly on budget
```

Off-by-one errors in loops

CONCEPT & REASONING

range(n) stops before n. For n=5, it produces 0, 1, 2, 3 and 4. Using n−1 omits the last index.

PYTHON EXAMPLE / CHECK

```
range(len(expenses) - 1)
```

Off-by-one errors in loops · applied

CONCEPT & REASONING

Test an asymmetric list whose last entry is non-zero. An all-zero list can hide some omission errors because the incorrect sum may still be zero.

PYTHON EXAMPLE / CHECK

```
[2,3,0,4,1]: wrong total 9; right
total 10
```

Check conditions at equality

CONCEPT & REASONING

The order and operators of if/elif/else matter. The first true branch is chosen.

PYTHON EXAMPLE / CHECK

```
if total >= allowance:
    status = "Over budget"
```

Check conditions at equality · applied

CONCEPT & REASONING

At total = allowance = 15,000, a first branch with >= incorrectly labels the student as over budget.

PYTHON EXAMPLE / CHECK

```
Correct equality outcome: Exactly on
budget
```

Count and average correctly

CONCEPT & REASONING

Use a count variable that grows when a zero is found. Do not reset the count after finding one. For averages, confirm the denominator matches the number of accepted values.

PYTHON EXAMPLE / CHECK

```
zeros += 1  # for each zero
average = total / 5
```

Count and average correctly · applied

CONCEPT & REASONING

With five values totalling 10,000, dividing by 4 gives 2,500, but the correct five-day average is 2,000.

PYTHON EXAMPLE / CHECK

```
[0,0,0,0,0]: count = 5
```

Reject invalid input and re-prompt

CONCEPT & REASONING

Validate the value, not just its type. Text like abc fails conversion. Negative numbers are numeric but invalid expenses. Only accepted values should be appended.

PYTHON EXAMPLE / CHECK

```
try:
    amount = float(text)
except ValueError:
    print("Use a number")
```

Reject invalid input and re-prompt · applied

CONCEPT & REASONING

Reprompt the same day after abc or -500; accept 0.0. Five accepted days require exactly five stored elements.

PYTHON EXAMPLE / CHECK

```
After rejection: do not append an
element
```

Testing functions in isolation

CONCEPT & REASONING

Test one function at a time so the source of a failure is easier to find. Use known inputs and outputs, then combine the functions in a full workflow.

PYTHON EXAMPLE / CHECK

```
def double(n):
    return n * 2
```

Testing functions in isolation · applied

CONCEPT & REASONING

A function that prints 6 but returns None will fail an equality check against 6.

PYTHON EXAMPLE / CHECK

```
double(3) == 6 → True
```

Simple automatic checks with assert

CONCEPT & REASONING

assert condition means “this should be true at this point”. A failing assertion highlights a violated expectation.

PYTHON EXAMPLE / CHECK

```
assert double(3) == 6
assert double(0) == 0
```

Simple automatic checks with assert · applied

CONCEPT & REASONING

Run several independent assertions. A passing assertion does not prove that every other case works.

PYTHON EXAMPLE / CHECK

```
AssertionError indicates a
failed check
```

Regression testing after a fix

CONCEPT & REASONING

Regression testing repeats earlier tests after a code change to ensure other cases still work.

PYTHON EXAMPLE / CHECK

```
T01 under | PASS
T02 equal | PASS
T03 over  | PASS
```

Regression testing after a fix · applied

CONCEPT & REASONING

After changing >= to >, rerun cases below, equal to and above the allowance. All three must pass.

PYTHON EXAMPLE / CHECK

```
A one-line fix can still cause
another failure
```

Compare numbers carefully

CONCEPT & REASONING

Computer floating-point values may be approximate. A formatted display is not proof of exact numeric equality.

PYTHON EXAMPLE / CHECK

```
round(2.0, 2)  # display is still
separate
```

Compare numbers carefully · applied

CONCEPT & REASONING

For this unit, compare the expected money values numerically and display results to two decimal places.

PYTHON EXAMPLE / CHECK

```
math.isclose(0.1 + 0.2, 0.3) →
True
```

Document a bug report

CONCEPT & REASONING

Evidence-rich reports make faults easier to reproduce and fix. Record the failing case before changing the code.

PYTHON EXAMPLE / CHECK

```
Bug: final item missing
Expected 60
Actual 30
```

Document a bug report · applied

CONCEPT & REASONING

The report distinguishes observed behaviour from suspected explanation. A suspicion is not a verified root cause.

PYTHON EXAMPLE / CHECK

```
Cause found: incorrect loop stop
value
```

Responsible use of help and tools

CONCEPT & REASONING

Own the evidence: runnable code, reproducible test outputs and explanations. Declare assistance and understand each changed line.

PYTHON EXAMPLE / CHECK

```
Test evidence must match actual
runs
```

Responsible use of help and tools · applied

CONCEPT & REASONING

Do not hide an unresolved test. Mark FAIL and explain the remaining limitation if one is not fixed by the deadline.

PYTHON EXAMPLE / CHECK

```
Assistance: tool/person/source and what it
helped with
```

Apply the full testing cycle

CONCEPT & REASONING

Test → observe → debug → fix → retest → report. Test outcomes must be connected to explicit requirements.

PYTHON EXAMPLE / CHECK

```
Problem → requirement → test → evidence
```

Apply the full testing cycle · applied

CONCEPT & REASONING

The final artefact is not only the corrected Python file: it also includes the test table and an explanation of how the repairs were verified.

PYTHON EXAMPLE / CHECK

```
Unit 6: diagnose, repair and
demonstrate
```

Worked test matrix

CONCEPT & REASONING

Required cases: normal under-budget, all-zero at zero allowance, exact equality, above allowance, and one rejected negative entry before accepting a valid amount.

APPLIED EXAMPLE / TAKEAWAY

Expected totals: 10000, 0, 15000, 22000, 6000.
Expected zero counts: 1, 5, 0, 0, 2.

Debugging with a real Python interpreter

CONCEPT & REASONING

A debugger pauses execution so variables and the next instruction can be inspected. Python’s built-in breakpoint() enters the default debugger, pdb, when enabled. Its n command executes the current line and stops at the next line; p expression displays a value; c continues; q exits. A breakpoint belongs before a suspected calculation.

EXAMPLE & RESULT

```
def total_cost(price, quantity):
    breakpoint()
    total = price + quantity
    return total

print(total_cost(1500, 3))
```

At (Pdb), p price gives 1500 and p quantity gives 3. n executes the incorrect addition. p total gives 1503; multiplication should give 4500.

Tracebacks and precise error messages

CONCEPT & REASONING

A traceback lists the chain of active calls leading to an exception. The last line identifies the exception type and message; the last relevant frame gives the file and line involved. The root cause may originate earlier, for example a value supplied by the caller.

EXAMPLE & RESULT

```
def area(width, height):
    return width * height

# A text value cannot multiply a float
area("5", 2.5)
```

TypeError. The function receives a string and a float; a numeric rectangle area requires numeric dimensions.

Recovering from predictable input errors

CONCEPT & REASONING

try/except handles a specific expected exception without terminating the entire programme. Conversion errors are different from invalid numeric ranges: float("abc") raises ValueError, while -5 converts successfully but may violate a requirement. math.isfinite() detects infinities and NaN.

EXAMPLE & RESULT

```
import math

while True:
    try:
        value = float(input("Amount: "))
    except ValueError:
        print("Enter a number")
        continue
    if math.isfinite(value) and value >= 0:
        break
    print("Enter a finite non-negative amount")
```

abc, -5 and nan are rejected; 0 and 12.5 are accepted. Only conversion ValueError is caught.

Repeatable automated checks

CONCEPT & REASONING

A table of test inputs and expected outcomes can be executed in a loop. Each assertion expresses a requirement. A failed assertion reports the case that did not match. Assertions are useful for tests, not as the sole validation mechanism for production inputs because optimisation can disable them.

EXAMPLE & RESULT

```
def cost(price, quantity):
    return price * quantity

cases = [(1500, 3, 4500), (2000, 0, 0)]
for price, qty, expected in cases:
    assert cost(price, qty) == expected
print("All tests passed")
```

All tests passed. Replacing * with + would fail the first case.

Focused fixes and version history

CONCEPT & REASONING

A debugging record links a reproducible failure to its cause, correction and retest. Version history preserves the before-and-after code and a concise change description. A regression test covers a fault already found so later changes cannot silently reintroduce it.

EXAMPLE & RESULT

```
Requirement: 5 daily amounts included
Failure: final day omitted
Cause: range(4) for five items
Fix: iterate over all values
Retest: [1, 2, 3, 4, 5] → 15
Commit: Include fifth day in total
```

A documented correction is stronger evidence than a screenshot of a successful run alone.

AI as a debugging assistant

CONCEPT & REASONING

An AI explanation is a hypothesis, not proof. A useful request includes the relevant code, the expected result, the actual result and the exact error message, without private data. A focused correction is checked against an independent expected outcome and related regression cases.

EXAMPLE & RESULT

```
Example prompt:
"My function returns 1503 instead of 4500
for price=1500 and quantity=3. Explain
the likely error and suggest one test.
Do not rewrite the whole programme."
```

The learner checks the arithmetic, changes + to *, reruns the failing case and verifies other cases.

Assignment submission

CONCEPT & REASONING

Correct the supplied flawed programme and submit runnable Python code, a concise bug report, a test table with expected and observed outputs, and an assistance declaration.

APPLIED EXAMPLE / TAKEAWAY

Use the course Google Form linked on the Activity U6 page.

Unit 6 · takeaways

CONCEPT & REASONING

The same test cases can check a repaired programme and reveal new faults. Reliable programmes communicate valid results and handle expected user mistakes clearly.

APPLIED EXAMPLE / TAKEAWAY

Python first: these testing habits transfer into Unit 7 with R.